Autonomous Robots (2025) 49:3
https://doi.org/10.1007/s10514-024-10187-z

Synthesizing compact behavior trees for probabilistic robotics domains

Emily Scheide[1] · **Graeme Best**[2] · **Geoffrey A. Hollinger**[1]

Received: 22 February 2024 / Accepted: 25 December 2024 / Published online: 14 January 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Complex robotics domains (e.g., remote exploration applications and scenarios involving interactions with humans) require encoding high-level mission specifications that consider uncertainty. Most current fielded systems in practice require humans to manually encode mission specifications in ways that require amounts of time and expertise that can become infeasible and limit mission scope. Therefore, we propose a method of automating the process of encoding mission specifications as behavior trees. In particular, we present an algorithm for synthesizing behavior trees that represent the optimal policy for a user-defined specification of a domain and problem in the Probabilistic Planning Domain Definition Language (PPDDL). Our algorithm provides access to behavior tree advantages including compactness and modularity, while alleviating the need for the time-intensive manual design of behavior trees, which requires substantial expert knowledge. Our method converts the PPDDL specification into solvable MDP matrices, simplifies the solution, i.e. policy, using Boolean algebra simplification, and converts this simplified policy to a compact behavior tree that can be executed by a robot. We present simulated experiments for a marine target search and response scenario and an infant-robot interaction for mobility domain. Our results demonstrate that the synthesized, simplified behavior trees have approximately between 15x and 26x fewer nodes and an average of between 8x and 13x fewer active conditions for selecting the active action than they would without simplification. These compactness and activity results suggest an increase in the interpretability and execution efficiency of the behavior trees synthesized by the proposed method. Additionally, our results demonstrate that this synthesis method is robust to a variety of user input mistakes, and we empirically confirm that the synthesized behavior trees perform equivalently to the optimal policy that they are constructed to logically represent.

Keywords Behavior tree · PPDDL · Synthesis · Generation · MDP

1 Introduction

The use of robots for repetitive and/or remote exploration applications is becoming increasingly feasible as advancements in autonomy become more widespread. Without the limitations associated with the requirement of teleoperation, robots can be deployed in greater numbers and for more complex tasks in applications ranging from marine monitoring (Molina-Molina et al., 2021; Ögren, 2012; Bhat et al., 2020) and subterranean exploration (Best et al., 2022) to infant-robot interaction for mobility (Helmi et al., 2022; Fitter et al., 2019; Vora et al., 2021). A control architecture can represent autonomy by respectively describing the association of sets of behavioral responses with certain world states. Manually designing control architectures requires expert knowledge and is more time-intensive the more complex the domain considered, e.g. a larger multi-robot team and higher uncertainty. Hence, an automatic method of control architecture synthesis that can accommodate domains with high degrees of current and future state uncertainty is required.

In robotics, common control architectures range from the well-known finite state machine to the more recent behav-

✉ Emily Scheide
scheidee@oregonstate.edu

Graeme Best
graeme.best@uts.edu.au

Geoffrey A. Hollinger
geoff.hollinger@oregonstate.edu

[1] Collaborative Robotics and Intelligent Systems (CoRIS) Institute, Oregon State University, Corvallis, OR 97331, USA

[2] Robotics Institute, School of Mechanical and Mechatronic Engineering, University of Technology Sydney, Ultimo, NSW 2007, Australia

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

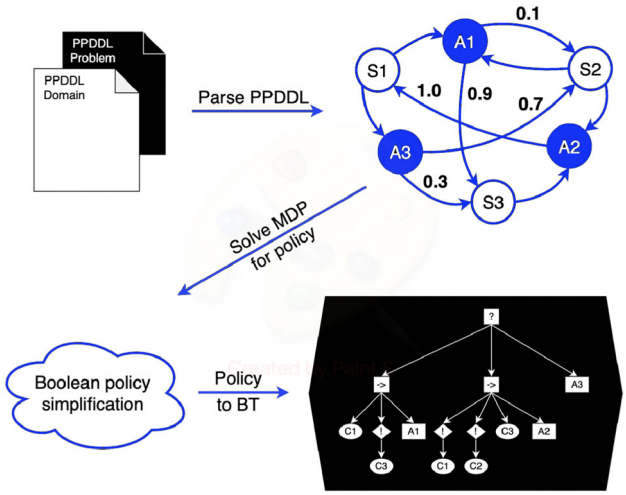


Fig. 1 An overview of our behavior tree synthesis algorithm, taking a PPDDL domain and problem specification as input and returning a simplified behavior tree with equivalent performance to the optimal policy (Color figure online)

ior tree, which is increasing in popularity (Colledanchise & Ögren, 2018; Iovino et al., 2022). Behavior trees have advantages over other common architectures, most notably in recursivity, modularity, and interpretability, because they are built with respect to tasks rather than states (Colledanchise & Ögren, 2018; Iovino et al., 2022; Ögren, 2012; Banerjee, 2018; Klöckner, 2023; Sprague et al., 2018; Gugliermo et al., 2023). The behavior tree is a directed rooted tree that is comprised of leaf (or execution) nodes that evaluate conditions or activate actions, and internal (or control flow) nodes that define a logic structure, which controls the flow of execution within the tree. The overall operation of a behavior tree occurs through the execution of tasks (via action nodes) in response to the changing world state (via condition nodes). This advantageous behavior tree structure and functionality makes behavior trees more feasible to manually design than other common architectures.

However, behavior tree manual design can still require an overwhelming amount of time and expert knowledge to be feasible for sufficiently complex domains. To accommodate the growing demand for autonomous robots, we require an adaptable and accessible automatic behavior tree synthesis method. Given a specification of a domain and problem that collectively describe a robot's capabilities and objectives, the goal of such an automatic synthesis method is to find the behavior tree that represents the optimal policy in the domain and for the problem specified. Additionally, this optimal behavior tree must be maximally interpretable to a user. There are a variety of methods that include behavior tree synthesis at least to an extent (see Sec. 2). However, these methods are often limited in domain applicability, in their ability to fully synthesize an entire behavior tree, because they do not account for nondeterministic action effects, and because they require high amounts of user input or execution time. Even increasingly popular large language models that allow for flexible, natural language input suffer from these disadvantages and cannot guarantee that generated behavior trees will be executable, interpretable or optimally-performing (Ao et al., 2024).

In this paper, to fill the existing gap in behavior tree synthesis, we present an automatic method of behavior tree synthesis that explicitly accounts for challenges associated with nondeterministic action effects, while maintaining behavior tree benefits like interpretability, and without limiting the scope of applicable domains. The Probabilistic Planning Domain Definition Language (PPDDL) (Lesner & Zanuttini, 2011) provides a generalizable and flexible way to specify robot capabilities and mission objectives for a broad scope of applications, including those with nondeterministic actions. Our method allows the user to specify a Markov decision process (MDP) in the form of a PPDDL domain and problem specification. This is in contrast with the specification of an MDP through the definition of the transition probability and reward matrices. Although MDPs in matrix form are more directly solvable, their manual definition is error-prone and time-intensive even for experts. PPDDL specification of an MDP has a lower risk of error and requires less time due to extensive PPDDL documentation (Lesner & Zanuttini, 2011). Given this PPDDL input, our method is able to return a simplified behavior tree that represents, and has equivalent performance to, the optimal policy for the MDP represented by the specification. The main components of the method are the following:

1. Parsing the PPDDL specification and converting it to the equivalent but solvable form of the MDP, i.e., the transition probability and reward matrices,
2. Given the MDP matrix form, solving the MDP for a policy, e.g. using value iteration,
3. Simplifying the policy to increase the interpretability of the final behavior tree by representing the policy as a Boolean sum of products expression and returning the essential prime implicants of the expression,
4. Converting the simplified policy to a behavior tree.

These phases allow our method to produce simplified, high-performing behavior trees with minimal human input, while maintaining applicability to complex domains with high levels of uncertainty. The key novelty is rooted in the PPDDL specification to MDP matrix conversion that enables the leveraging of prevalent MDP solvers, the application of Boolean simplification to policies for their conversion to simplified behavior trees, and the details of the behavior tree synthesis method that focuses on behavior tree compactness and interpretability while maintaining equivalent function to

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

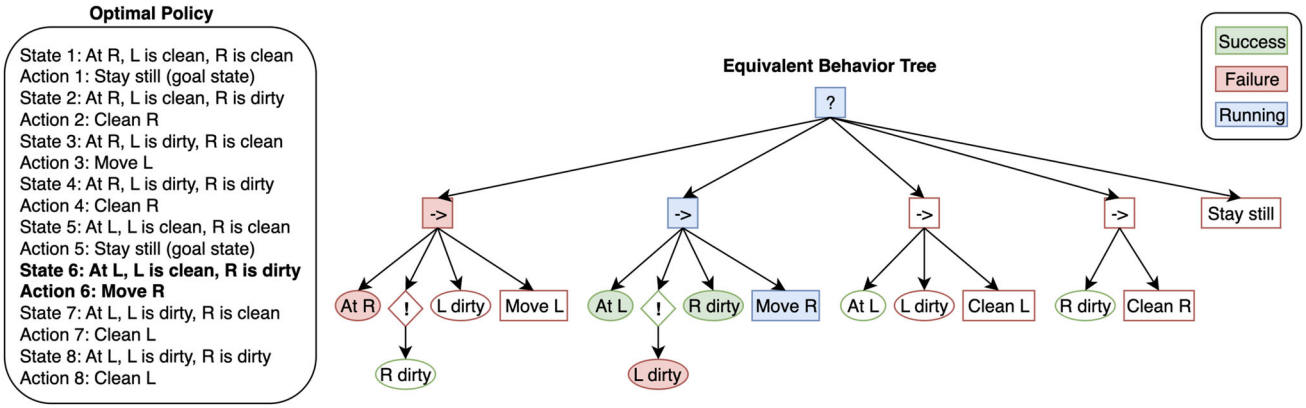


Fig. 2 For the 2-cell vacuum cleaner toy problem (Russell & Norvig, 2010), we show the optimal policy and the equivalent synthesized behavior tree. Both the policy and the behavior tree are in the same state, and they are accordingly executing the same action. Node statuses are denoted by color (see upper right key) (Color figure online)

the simplified policy. We illustrate the functional equivalence of the optimal policy to the synthesized behavior tree for the 2-cell vacuum cleaner toy problem (Russell & Norvig, 2010) in Fig. 2.

To demonstrate the efficacy of our method, we consider two distinct robotics applications that present unique challenges for behavior tree synthesis. The first is marine target search and response, and the second is robotic intervention for infant mobility. We assess our method's performance in each of these domains by evaluating the synthesized behavior tree in several ways. First, we evaluate the synthesized behavior tree with respect to compactness (minimized number of nodes) and execution efficiency (minimized average number of active conditions for selecting the active action). Results show that our method is crucial to maximizing the compactness and efficiency of the synthesized behavior tree. Second, we vary the level of PPDDL domain complexity to evaluate scalability and compare against a baseline domain that is identical except robot action effects are deterministic. Results show that our method achieves a higher average reward with the full PPDDL specification than the deterministic baseline, providing a greater increase in reward the more complex the domain is. Third, we evaluate the impact of different numbers and types of mistakes being present in the input PPDDL specification, and we find that our method is still able to synthesize valid behavior trees with graceful performance degradation (if any) in the majority of cases. Finally, we empirically confirm that the synthesized behavior trees perform equivalently to the optimal policy in the given MDP, and we report the time required for synthesis for varying domains. Overall, these evaluations demonstrate that our method is capable of synthesizing a behavior tree that has equivalent performance to the optimal policy in the domain specified, while prioritizing compactness, the related benefit of interpretability, robustness to user input errors, and minimal required running time.

2 Related work

2.1 Behavior trees

Behavior trees are a recent, yet popular control architecture that were introduced first in the video game industry (Colledanchise et al., 2018), which are now widely used in robotics applications ranging from marine monitoring (Ögren, 2012; Bhat et al., 2020) to human-robot interaction (Tulathum et al., 2022; Axelsson & Skantze, 2019). This popularity stems from the benefits that behavior trees possess in comparison with other control architectures such as finite state machines or decision trees (Colledanchise & Ögren, 2018). These benefits most notably include scalability, recursivity, modularity, and interpretability (Colledanchise & Ögren, 2018; Iovino et al., 2022; Ögren, 2012; Banerjee, 2018; Klöckner, 2023; Sprague et al., 2018; Gugliermo et al., 2023). Scalability is important for the use of behavior trees in complex domains. Recursivity is important for the re-execution of behaviors without replanning. Modularity is vital for enabling the logic of a behavior tree to be updated without sacrificing functionality. Interpretability is crucial for user understanding of and trust in robot autonomy.

2.1.1 Offline behavior tree synthesis

These qualities make behavior trees particularly well-suited for automatic synthesis. Offline behavior tree synthesis methods are often based in reinforcement learning methods, such as genetic algorithms (Styrud et al., 2022; Ma et al., 2023; Colledanchise et al., 2018; Lim et al., 2010; Neupane &

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Goodrich, 2019; Perez et al., 2011; Styrud et al., 2022), Q-learning (Banerjee, 2018; Dey & Child, 2013), Monte Carlo learning methods (Scheide et al., 2021), and others such as Kartašev et al. (2021), Ao et al. (2024). However, these current methods are lacking in that they suffer from one or more of the following limitations: they focus on low-level action learning as opposed to full behavior tree synthesis, are overly domain-specific, do not explicitly account for nondeterministic actions effects, require a high level of specific user input, or require an infeasible amount of learning time.

Large language models (LLMs) are becoming increasingly popular due to their ability to flexibly take in natural language input, and they have initially been examined in the context of behavior tree generation (Ao et al., 2024). However, Ao et al. (2024) demonstrates that the price of having this flexibility is a requirement of human-in-the-loop assistance for the best behavior tree generation results, and even then no guarantee that the returned behavior tree will be executable, let alone optimally-performing based on the input scenario. This further demonstrates the need for a behavior tree synthesis method that can provide these guarantees, while minimizing the need for potentially expensive, time-intensive, and error-prone human intervention.

2.1.2 Online behavior tree synthesis

Online behavior tree synthesis methods (Safronov et al., 2020; Gao et al., 2021; Jamal & Panov, 2021; Wu et al., 2021) inherently bypass issues with computation time, but still suffer similarly from the other limitations we see with offline behavior tree synthesis methods. However, because the behavior tree is being synthesized online, interpretability prior to execution is not possible, so user understanding is more likely to suffer. For example, although (Safronov et al., 2020) explicitly accounts for uncertainty and nondeterministic actions, because synthesis is done online, the user is not able to interpret robot behavioral intention before actions begin executing. In contrast, our behavior tree synthesis method avoids these issues. Our method harnesses the flexibility and generalizability of PPDDL to minimize the threshold of entry for the user, and maximize the scope of domain applicability and breadth of behavior tree design. Additionally, our method specifically focuses on maximizing user understanding, i.e., interpretability. It is important to note that behavior tree compactness, i.e., the number of nodes in a given tree, is a common metric for the evaluation of behavior tree interpretability (Gugliermo et al., 2023; Scheper et al., 2016; Gustavsson et al., 2022; Robertson & Watson, 2015; Wathieu et al., 2022). Having fewer conditions has also been associated with increased behavior tree efficiency (Gugliermo et al., 2023).

2.2 Probabilistic PDDL

In order for a synthesis method to be as applicable to as many domains as possible, it is crucial that the input not only avoid domain specificity, but also that it is feasible in terms of the required user knowledge and time commitment. The Planning Domain Definition Language (PDDL) is a well-known and documented language that is used to specify domains and problems (Constructions Aeronautiques, 1998). Probabilistic PDDL (PPDDL) is an extension that allows for the specification of domains with nondeterministic action effects (Younes & Littman, 2004). PPDDL is well-suited to specify a Markov decision process (MDP), which is advantageous due to the prevalence of MDP solvers. The use of PPDDL is also key to our method's breadth of applicability, and low threshold of entry due to extensive and readily available documentation on PPDDL syntax. Despite this and the prevalence of MDP solvers, PPDDL specifications of MDPs cannot be direct input for most off-the-shelf MDP solvers. These solvers often expect MDPs to be given in the form of transition probability and reward matrices. There has been minimal work on solving MDPs specified in PPDDL without conversion to matrix form, such as Lesner and Zanuttini (2011); our method employs a unique specification to matrix conversion method, which avoids the complexity of the solver in Lesner and Zanuttini (2011).

2.3 Policy simplification

When we consider an MDP that has been solved for a policy, it is not given that that policy will be interpretable. This is often due to domain complexity, e.g. a large number of possible states with many components each, which is difficult to visually represent compactly enough for sufficient logical understanding. This understanding is crucial for a user to put trust in the solution they have received for the given MDP. However, there is a gap in the literature with respect to policy simplification for this purpose, particularly as it relates to harnessing the interpretability benefits of behavior trees for policy representation. Boolean logic factorization has been used with the goal of compact behavior tree learning from demonstration, but this method is limited by input requirements (Gugliermo et al., 2023).

Behavior trees have been used for the optimized execution of PDDL plans (Martín et al., 2021), but this method is limited in applicability. This is because, unlike our method, Martín et al. (2021) cannot handle nondeterministic action effects, PDDL constraints or negative preconditions, or logic that requires fallback or decorator nodes to equivalently represent in a behavior tree.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

3 Preliminaries

3.1 Behavior trees

A behavior tree is a directed rooted tree that is comprised of control flow nodes and execution nodes (Colledanchise & Ögren, 2018). Control flow nodes determine which areas of the behavior tree are active, and include *fallback* nodes ("or" logic, denoted by ?), *sequence* nodes ("and" logic, denoted by $\rightarrow$), *parallel* nodes (concurrent activity, denoted by ||), and customizable *decorator* nodes. A common example of a *decorator* is the *not-decorator* (denoted by !), which returns the logical complement of a single child *condition* node. Execution nodes are either *condition* nodes that represent things the robot can sense or *action* nodes that execute robot behaviors. All nodes can return statuses of *success*, *failure*, and *running*, except conditions which only return the former two. In this work, we consider *sequence*, *fallback*, *not-decorator*, *action*, and *condition* nodes.

3.2 PPDDL syntax

In this work, we employ the generally applicable and widely known Probabilistic Planning Domain Definition Language (PPDDL) (Younes & Littman, 2004) to specify the elements of an MDP. Formal PPDDL definitions are provided in (Younes & Littman, 2004), and the following is an overview of the relevant PPDDL syntax. Consider an MDP that is defined by a state space $\mathcal{S}$, an action space $\mathcal{A}$, transition probabilities $P_a(s, s')$, and rewards $R_a(s, s')$ $\forall a \in \mathcal{A}$ and $\forall s, s' \in \mathcal{S}$. This MDP pertains to a robot that has acting capabilities described by $\mathcal{A}$, and sensing capabilities described by a set of conditions $\mathcal{C}$.

A PDDL domain includes specification of `requirements`, `types`, `predicates`, and `actions`. Each requirement denotes a genre of syntax that needs to be supported in that domain. A type is used to categorize a PDDL term such as an object or variable. Optionally, `constants` and `constraints` can also be defined. Each condition $c \in \mathcal{C}$ is defined as a predicate, and a constraint can be used to denote the allowed relationship between any predicates that would otherwise contradict. Each action $a \in \mathcal{A}$ includes `parameters` (if any), a `precondition`, and an `effect`. Parameter definition is domain-dependent and can often be subjectively paired with or replaced by the use of constants. An action `precondition` and `effect` respectively represent the state before (s) and after (s') the action occurred, and each is defined by a series of predicates. However, only deterministic actions can be specified in a PDDL domain.

A PPDDL domain (see Appx. A: Fig. 11, Fig. 14) extends a PDDL domain with support for the `probabilistic-effects` and `rewards` requirements, i.e. the `mdp` requirement collectively. This allows for an action `effect` to represent nondeterministic state transitions, i.e. multiple end states s', each associated with a transition probability $P_a(s, s')$ and a reward $R_a(s, s')$. This reward can be positive or negative, based on whether the action was effective or not.

A problem in PPDDL (see Appx. A: Fig. 13, Fig. 16) includes specification of the associated domain by name, the relevant `objects`, initialization of the world state with predicates in `init`, and a `goal` for syntactical completeness. In this work, the `goal` specification is irrelevant to the specified MDP and its solution, because the actual goal is implicitly to maximize reward.

Thus, a domain and problem specification in PPDDL can encompass all of the information required to define an MDP. This allows users to take advantage of the accessibility of PPDDL syntax documentation, but further manipulation is required to obtain an MDP representation that is conveniently solvable, e.g., transition probability and reward matrices. MDP solvers are commonly set up to take these MDP matrices as input, but these matrices are non-trivial to create manually due to the likelihood of human error and inherent time intensiveness. Our method avoids these issues by bridging the gap between a PPDDL specification of a given MDP and the equivalent matrix form automatically.

4 Problem formulation

We address the problem of automatically synthesizing a behavior tree from a user-defined PPDDL domain and problem specification that contains all of the components of a Markov decision process (MDP), but is not yet in a solvable MDP matrix form. PPDDL is formally defined in Younes and Littman (2004), and we provide full PPDDL specifications for the distinct scenarios we simulate for the evaluations of our method in Appendix A.

The preliminary goal is to solve the MDP for a policy that maximizes the robot's task performance, e.g., successfully finding and manipulating objects or eliciting certain positive behaviors from human agents. The ultimate goal is to represent the optimal policy as a simplified behavior tree through the appropriate aggregation of conditions with actions. We formalize this problem as follows.

Formally, given the PPDDL domain and problem specification, the goal is to synthesize the most compact behavior tree b^* that equivalently represents the optimal policy π^* in the represented MDP. Policy π^* maximizes the expected reward as defined by the specification, i.e.,

$$\pi^* = \underset{\pi \in \prod}{\operatorname{argmax}} \mathbb{E}\left[\sum_{t=0}^{T} R_{a_t}\left(s_t, s_{t+1}\right)\right] \tag{1}$$

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Algorithm 1 PPDDLtoBT: Overview of behavior tree synthesis

Input: PPDDL domain and problem specification $\mathcal{S}$
Output: simplified behavior tree b
▷ Convert to MDP matrix form
1: $P, R \leftarrow$ PPDDLtoMDP(S)
▷ Solve MDP using Value Iteration
2: $policy \leftarrow$ MDPtoPolicy(P, R)
▷ Boolean policy simplification
3: $\pi^* \leftarrow$ SIMPLIFY($policy$)
▷ Convert simplified policy to BT
4: $b \leftarrow$ GENERATEBT(π^*)
5: **return** b

where $\prod$ is the set of all possible policies, T is a time horizon, and a_t is the action given by the policy, $\pi(s_t)$. The goal is also for b^* to be as compact as possible to make it more interpretable by a user regardless of their behavior tree experience. In other words, in order to increase interpretability, we aim to maximize compactness. We define several measures of compactness in Sec. 6.

5 PPDDLtoBT: behavior tree synthesis algorithm

We propose a new method of behavior tree synthesis comprised of four main components, as shown in Fig. 1 and initially detailed in Alg. 1. The method begins with a user-defined PPDDL specification of a domain and problem that collectively represent an MDP. Next, this input is passed to Alg. 2, which parses the PPDDL specification and converts it into a solvable and equivalent MDP form, i.e., a transition probability matrix and a reward matrix. Given the MDP in matrix form, it is then solved for a policy using value iteration. Then Alg. 3 takes the policy and simplifies it using Boolean algebra. Finally, Alg. 4 converts the simplified policy into the equivalent, but more compact and interpretable behavior tree (BT) representation. We detail the PPDDL specification to MDP matrix form conversion algorithm, the Boolean algebra policy simplification algorithm, and the policy to behavior tree synthesis algorithm as follows.

5.1 Conversion from PPDDL to MDP matrices

Given the user-defined PPDDL domain and problem specification that represents an MDP, the goal of this phase of the method (Alg. 1, line 1) is to generate a transition probability matrix P and a reward matrix R that collectively represent the equivalent MDP, but in a more solvable form (i.e., a form that can be passed to commonly used MDP solvers without further manipulation).

In Alg. 2, we achieve this by first parsing the PPDDL specification such that the specified information can to be converted into all possible states and actions in the given domain (line 3). In order to allow our method to parse a broader range of PPDDL specifications, we extend an existing parsing method (Bueno, 2017) that without extension can predominantly handle the following PDDL requirements: `strips`, `typing`, `equality`, `probabilistic-effects`. Our extension provides handling of syntax necessary for the following: the requirements `rewards` and `mdp`, and the PPDDL terms `constants` and `constraints`. The states that this parsing method returns are determined through assessment of the `predicates` and `types` terms, in addition to the `constants` and `constraints` terms when applicable. The actions are explicitly listed in the PPDDL domain, so each action term is simply individually parsed.

Algorithm 2 Converting PPDDL to MDP transition probability and reward matrices

1: **function** PPDDLtoMDP(S)
2: $P, R \leftarrow (), ()$
3: $states, actions \leftarrow$ PARSEPPDDL(S)
4: $N \leftarrow num_states$
▷ Determine action effects for all actions
5: **for** a in $actions$ **do**
6: $p_a, r_a \leftarrow zeros(N, N), zeros(N, N)$
7: $effects \leftarrow \{\}$
8: **for** $i = 0$ to N **do**
9: $s_{start} \leftarrow states[i]$
10: **if** PRECONDITIONS(s_{start}, a) **then**
11: $[s_{end}, p_i, r_i] \leftarrow$ GET(s_{start}, a)
12: **else**
13: $[s_{end}, p_i, r_i] \leftarrow [s_{start}, 1, 0]$
▷ Get end state, probability, reward
14: $effects \leftarrow effects \cup \{[s_{end}, p_i, r_i]\}$
▷ Group effects with same end state
15: $M \leftarrow$ STATEMATCHES($effects$)
▷ Combine matching effects
16: **for** $group$ in M **do**
17: $j \leftarrow$ GETSTATEINDEX($group$)
18: **for** $effect$ in $match_group$ **do**
19: $p_j, r_j \leftarrow effect$
20: $p_a[j, i], r_a[j, i] \mathrel{+}= p_k, r_k$
21: $P, R \leftarrow$ ADDSUBMATRIX(p_a, r_a)
22: **return** P, R

Next in Alg. 2, each possible action a is examined with the goal of generating the transition probability submatrix p_a and the reward submatrix r_a. These submatrices are the portions of P and R that pertain to a, i.e., the likelihoods of each possible end state if action a is taken in each possible start state and the associated rewards respectively (lines 5-22). Each action a has preconditions that must be true in order for the action to be executable in a given state s. If the preconditions of a are true in the ith state s_{start} (line 10), then each outcome of taking a in s_{start} is generated in the form of an end state s_{end}, the probability that s_{end} will occur p_i, and the associated reward r_i (lines 8-14). When s_{end} is the same for multiple outcomes, the associated end state probabilities and rewards are respectively summed (lines 15-

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Algorithm 3 Boolen algebra policy simplification

```
1: function SIMPLIFYPOLICY(policy)
2:     C ← GETCONDITIONSYMBOLS()
3:     S ← GETVALIDNUMERICALSTATES()
       ▷ Define invalid numerical states
4:     D ← GETDONTCARES()
5:     for a in policy.actions do
6:         M ← GETMINTERMS(a)
7:         SOP ← SUMOFPRODUCT(C, M, D)
8:         policy_s ← SIMPLIFYSOP(SOP)
9:         D ← D ∪ M                    ▷ Ignore prior minterms
10: return policy_s
```

21). All outcomes are then used to appropriately update p_a, which is added to P, and the same occurs for r_a with R (line 21). This process repeats for all actions, and returns the resulting solveable MDP representation in the form of P and R. Given P and R, the next step is to solve the MDP for a policy.

5.2 Boolean algebra policy simplification

By passing P and R to a value iteration algorithm, the MDP is solved for the optimal policy π^*. However, this policy is not in its simplest form, which presents a challenge in that it largely sacrifices the crucial interpretability benefit of the equivalent behavior tree (Sec. 5.3). Without further simplification, this behavior tree would be comprised of as many subtrees as there are states in the policy. This would fail to take advantage of the fact that behavior trees can be built with respect to tasks (i.e., actions) as opposed to states; a feature that allows behavior trees to be more compact than other autonomy representations or control architectures. To increase the interpretability of the behavior tree representation of the policy, π^* is simplified to π_s^* using Alg. 3. This simplification has no effect on policy performance because functionally π_s^* is equivalent to π^*, so both achieve the maximum reward.

Alg. 3 uses Boolean algebra simplification techniques based on the Quine-McCluskey algorithm (Quine, 1952), which deterministically finds the minimal form of a Boolean expression. First, all conditions are represented as *symbols* in C, and the state representation is simplified to numeric form, i.e. 0 for all false conditions and 1 for all true conditions (line 2). Only some numeric states are valid given condition constraints defined in the original PPDDL specification. By parsing the PPDDL for constraint information, the valid numeric states S are separated from the invalid, or *dontcares* D, which will be ignored to improve simplification (lines 3-4).

Next, each action a that appears in the policy, π^*, is used to evaluate the π^* for redundancy in the order that the actions appear in the PPDDL specification (lines 5-10). All numeric states in S that are associated with a are grouped as *minterms* M (line 6). This allows the problem to be represented as a sum of products expression (line 7) that can then be simplified into disjunctive normal form. This is done using the laws of Boolean algebra to find the essential prime implicants of the expression that can then be used to update the simplified policy π_s^* (line 8). This process repeats for all actions in π^*, disregarding previously evaluated *minterms* by adding them to D after each action evaluation round (lines 9). Ultimately, the simplified policy, π_s^*, is returned, comprised of a logic expression describing the set of states in which an action should be taken for each unique action in π^*. It is this policy, π_s^*, that can then be represented as a simplified behavior tree.

5.3 Behavior tree synthesis from a simplified policy

Finally, Alg. 4 converts the simplified policy π_s^* into a behavior tree b^*. The behavior tree is initialized with a *fallback* node at the root (lines 2-3). A subtree with a *sequence* node at its root is created for every element in π_s^*, given that each element is pairing of a logic expression, l, and an action, a (lines 4-5). Each l corresponds to only a single state before policy simplification, but can represent and often represents a group of states as a result of simplification. When a logic expression represents a group of states, these states are related because of their association with the same action in the original policy, π^*.

Depending on whether the logic expression is solely conjunctive (*and* logic) or involves disjunction (*or* logic), a different hierarchy of nodes are added to the corresponding *sequence*-rooted subtree. In the *and* case, a series of *condition* and *not-decorator* nodes that describe the logic expression from the current $l-a$ pair are added beneath the subtree root, i.e., *sequence* node (lines 6-8, 15-23). If a condition is true in that expression, a *condition* node is added as is, while a *not-decorator* node is added with a *condition* node as its child to denote conditions that are false in that expression (lines 15-23). The *or* case could be handled similarly by creating a subtree with this same structure for each term in the disjunction l. However, further simplification can be done to obtain a more compact subtree if l is disjunctive (lines 9-11, 24-32). This additional simplification is achieved through the separation of conditions that appear in all l terms (i.e. the conditions that are common factors, C_o) from the rest (line 10). This allows for the new subtree to be assembled with the common conditions added first beneath the *sequence* node root (lines 11, 25), followed by a *fallback* node that has a series of *sequence* nodes beneath it (lines 26-29). Beneath each of these *sequence* nodes are the conditions left in each term of l after the common conditions were factored out (lines 30-31).

For either type of logic expression, l, the final step in creating a subtree with equivalent logic to a given $l-a$ pair in the simplified policy is to add the action from that pair as

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

the final child node of the subtree root *sequence* (line 12). Thus, the *action* node will only be executed in the correct or one of the correct states.

This synthesis process results in behavior trees with a *fallback* root node that has either *sequence* or *action* nodes immediately beneath it in the first layer. In the second layer, there can be *condition*, *not-decorator*, *fallback*, and *action* nodes. In the third layer, there can be *sequence* nodes beneath second-layer *fallback* nodes, and *condition* nodes beneath second-layer *not-decorator* nodes. All nodes in the fourth layer are beneath third-layer *sequence* nodes, and are either *condition* or *not-decorator* nodes. The *condition* nodes beneath any fourth-layer *not-decorators* are the only nodes in the fifth and final layer.

5.4 Algorithm analysis

By construction, the synthesized behavior tree b^* represents, and therefore has equivalent performance in simulation to, the optimal policy for the MDP specified in PPDDL as input. In other words, b^* contains all the logical state-action mappings in the policy, simply in a more compact form. For the domains we have examined, the execution time from input to returning b^* ranges from $<$ 1 second to approximately 7 minutes, depending on domain complexity, e.g. number of states and actions. The time complexities of Alg. 2 and Alg. 4 respectively are $\mathcal{O}(S \cdot A)$ and $\mathcal{O}(S \cdot C)$ given S states, A actions, and C conditions. The MDP solver we use, i.e. value iteration, has a time complexity of $\mathcal{O}(N \cdot S^2 \cdot A)$ where N is the number of iterations. The time complexity of Alg. 3 is $\mathcal{O}(A_p \cdot QMC)$ where there are A_p actions in the policy, and QMC is the exponential time complexity of the Quine-McCluskey algorithm (Quine, 1952). Thus, the Alg. 1 time complexity (the sum of the others) is dominated by QMC. We observe this empirically in that the approximate average percentage of the total runtime was 99% for Alg. 3, and <1% each for Alg. 2, the MDP solver, and Alg. 4.

Algorithm 4 Synthesizing a behavior tree from a simplified policy

```
1: function CREATEBT(policy_s)
2:     b ← BehaviorTree('')
3:     b.root ← Fallback()                                  ▷ BT root
       ▷ For each logic expression-action pair
4:     for l, a in policy_s do
5:         s ← Sequence()                                   ▷ Subtree root
6:         C ← [c in l]                                     ▷ Get conditions in l
7:         if TYPE(l) == AND then
8:             s ← ADDCONDITIONS(C, s)
9:         if TYPE(l) == OR then
           ▷ Factor out conditions, save others
10:            C_o, C_l ← COMMONCONDITIONS(l)
11:            s ← ORSUBTREE(C_o, C_l, s)
           ▷ Add action node
12:        s.children ← s.children ∪ a
13:        b.root.children ← b.root.children ∪ s
14: return b
15: function ADDCONDITIONS(C, n)
16:     for c in C do
17:         if c.value = 1 then                              ▷ If True
18:             n.children ← n.children ∪ c
19:         else                                             ▷ If False
20:             d ← NotDecorator()
21:             d.child ← c
22:             n.children ← n.children ∪ d
23:     return n
24: function ORSUBTREE(C_o, C_l, n)
25:     n ← ADDCONDITIONS(C_o, n)                            ▷ Factored
26:     f ← Fallback()
27:     n.children ← n.children ∪ f
28:     for C in C_l do                                      ▷ Lists of non-factored
29:         s ← Sequence()
30:         s ← ADDCONDITIONS(C, s)
31:         f.children ← f.children ∪ s
32:     return n
```

6 Experiments

We designed two distinct application domains to evaluate the performance (e.g., compactness, reward, and runtime) of our method given varying degrees of uncertainty and scale. The first is a marine target search and response domain (see Sec. 6.2), and the second is an intervention for infant mobility domain (see Sec. 6.3).

It is important to note that there are no comparable behavior tree synthesis methods in the existing literature. As discussed in greater detail in Section 2, there are methods that handle subsets of what our method does, such as solving MDPs specified in PPDDL (Lesner & Zanuttini, 2011), using boolean logic to increase behavior tree compactness (Gugliermo et al., 2023), using behavior trees to execute PDDL plans (Martín et al., 2021). Other methods exist that can generate behavior trees, e.g., using a simulator, learning from demonstration, or other method requiring input that is not feasible to specify in PPDDL (see Section 2). None of these methods can take in the same input as our method and return a behavior tree that we could then compare our synthesized behavior tree's performance against.

6.1 Overview of evaluations

We first conduct the following two evaluations: 1) an ablation study that focuses on the compactness and relative node activity of the synthesized behavior tree, and 2) a comparison of the reward achieved by the synthesized behavior tree for versions of each domain defined by varying action effect uncertainty (likelihood of action success) and penalty magnitudes (negative rewards for action failure). Input for these evaluations is a domain-specific problem specification and either a PPDDL specification P_d or a specification D_d that

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

is identical except robot action effects are deterministic, and where d denotes the domain.

Next, we conduct a sensitivity analysis to determine the effect of mistakes in the input PPDDL MDP representation on the performance of the synthesized behavior tree. Then, for a range of domain variations, we compare the performance of the synthesized behavior tree with the performance of the optimal policy it represents in the associated MDP. This empirically demonstrates the logical equivalence of the synthesized behavior tree and the optimal policy. Lastly, we report the total behavior tree synthesis runtime for our method for each domain variation.

6.1.1 Behavior tree compactness evaluation

First, to conduct the ablation study, we define comparison methods by removing components of the proposed method to demonstrate the benefit they bring when combined with the other components. These comparison methods are defined by their components that differ from the proposed method as follows:

1. **PPDDLtoBT:** our proposed method.
2. **Keeping don't cares:** Simplification is not expedited by removing *dontcares*, e.g. invalid numeric states.
3. **Without simplification:** Synthesizing a behavior tree from a policy that has not been simplified.

For each of these four methods, given P_d or D_d as input, we evaluate the synthesized behavior tree's compactness and execution efficiency in the following ways:

1. n_a: total number of action nodes,
2. n_c: total number of condition nodes,
3. n: total number of nodes of any type,
4. F_n: scaling factor representing the n ratio of a given method against the proposed method,
5. $\overline{N}_{c/a}$: average number of active conditions for selecting the active action,
6. $F_{c/a}$: scaling factor representing the $\overline{N}_{c/a}$ ratio of a given method against the proposed method.

The number of nodes, n, in a behavior tree (BT) is a common metric used to evaluate compactness and potential for interpretability (Gugliermo et al., 2023; Scheper et al., 2016; Gustavsson et al., 2022; Robertson & Watson, 2015; Wathieu et al., 2022). We also specifically examine the number of action and condition nodes, n_a and n_c, to further inform the same criteria. The number of condition nodes has been associated with higher efficiency in behavior tree execution as well (Gugliermo et al., 2023). We examine compactness, efficiency in BT execution and the potential for BT interpretability to a further extent by also considering relative node activity. When an action is executed (i.e., becomes active in a BT) certain preconditions must be true (i.e., certain condition nodes must be active and return the appropriate status, either true or false). By definition, given that we do not consider concurrent action execution through the use of parallel nodes, only one action can be active and executing in a BT at a given time. We define $\overline{N}_{c/a}$ as the average number of conditions that must be active for a given action to be independently active. It follows that minimizing $\overline{N}_{c/a}$ suggests that a BT is likely to be more efficient and interpretable. A lower $\overline{N}_{c/a}$ means fewer conditions must be ticked for an action to be executed, and in the same way fewer conditions must be looked at by a user to understand why an action is occurring.

6.1.2 Behavior tree reward evaluation for probabilistic and deterministic domains

Second, we compare the reward achieved by the behavior tree synthesized by the method given a deterministic baseline specification D_d to that given a PPDDL specification P_d, where d denotes the domain. This comparison shows the importance of our method's ability to take input that models uncertainty in action effect, i.e., how this ability leads to broader method applicability and higher performance (as measured by reward) than would be possible if only deterministic input was permissible.

For each domain in this comparison, we vary two parameters: action effect uncertainty, i.e. the likelihood that each action will fail, and penalty magnitude, i.e. the consequence associated with false negative and false positive action outcomes. Details, such as action effect uncertainty and penalty magnitudes, are specified in the PPDDL domain and problem the user provides (see Appx. A). For each domain-parameter combination, we carry out 100 trials to obtain average reward values that we report as a percent difference between the average reward achieved given P_d and that given D_d.

Most deterministic domains should be possible to specify in PDDL, just as most domains with probabilistic actions, e.g., MDPs, should be possible to specify in PPDDL. Demonstrating our method's capability of synthesizing behavior trees with equivalent performance to the optimal policy given a domain described in either PDDL or PPDDL provides further evidence of the method's broad domain applicability. Similarly, the parameter exploration aspect of this evaluation demonstrates that our method does not begin to fail as the frequency of action failures and associated penalty increase.

6.1.3 Sensitivity analysis given PPDDL mistakes

Another potential source of failure is mistakes in the input PPDDL. The behavior tree synthesized by our method will

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

perform equivalently to the optimal policy in the MDP this PPDDL input represents. However, if this MDP representation is inaccurate, this policy and thus the synthesized behavior tree may not perform optimally with respect to the actual MDP. To examine the effect of different inaccuracies in the MDP representation, we consider the following types of mistakes, for each of which one or more action is affected as follows:

1. **Missing precondition:** it is missing one correct precondition.
2. **Added precondition:** it has an additional incorrect precondition.
3. **Higher reward:** it has a reward with a magnitude increase of 1.
4. **Higher penalty:** it has a penalty with a magnitude increase of 1.
5. **Lower likelihood:** it has a 10% lower likelihood of action success.
6. **Higher likelihood:** it has a 10% higher likelihood of action success.

Given the accurate PPDDL specification, we inject the following numbers of each of these types of mistakes: 1) one in a single action, 2) one in each of approximately half of the actions, and 3) one in all possible actions. The set of possible actions is defined by the domain and type of mistake. For each type of mistake and number of mistakes injected, we evaluate the reward obtained by the synthesized behavior tree in the actual MDP.

6.1.4 Policy and behavior tree equivalence evaluation

Our method is constructed such that the optimal policy in the given MDP should always perform equivalently to the behavior tree synthesized from it. In this evaluation, we demonstrate this equivalence by comparing the reward obtained by the policy and the reward obtained by the synthesized behavior tree. We conduct this evaluation in 20 variations of each domain, where these variations are each defined by varying the action effect uncertainty (likelihood of action success) or penalty magnitudes (negative rewards for action failure). Note that these are the same probabilistic domain variations we examined in the compactness evaluation described in Section 6.1.2.

6.1.5 Runtime report

For the same 20 probabilistic domain variations we examined for the reward evaluation (Sec. 6.1.2) and the equivalence evaluation (Sec. 6.1.4), we report the time it takes our method to synthesize the final behavior tree. This demonstrates the consistency of our method's performance and informs the feasibility of applying our method to other domains.

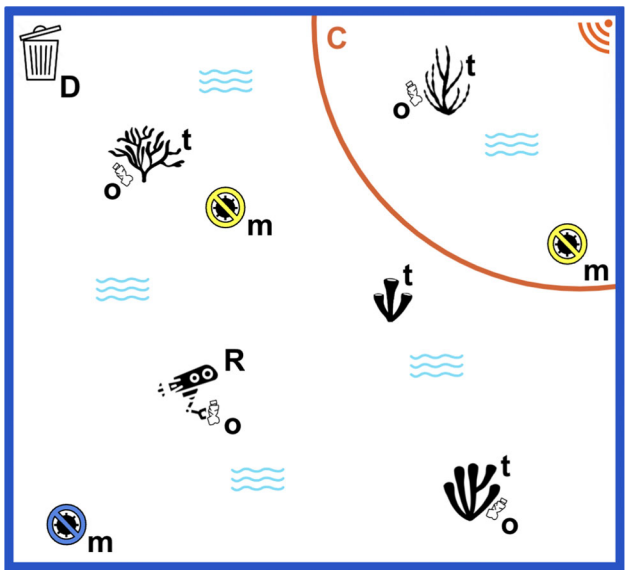


Fig. 3 A simplified visualization of the marine target search and response domain where letters (i.e., R, D, C, o, t, and m) denote objects and locations, and colors (i.e., orange, yellow, and blue) provide additional details. The robot R is shown moving away from a mine m (blue if disarmed, yellow otherwise), picking up an object o (e.g., trash) that needs to be taken to the drop off location D. Other tasks include identifying target locations t (e.g. coral), and moving to within communication range C (orange) to report them (Color figure online)

6.2 Marine target search and response

6.2.1 Experimental setup

We present a marine target search and response scenario, inspired by Scheide et al. (2021) and visually represented in Fig. 3. The robot in this scenario can explore the environment, retreat to within communication range, find and report the location of objects of interest (e.g., certain species of coral), pick up and drop off other objects (e.g., trash), and disarm mines. The goal is to carry out these actions such that the reward is maximized. Actions associated with a unique reward magnitude, in order of increasing magnitude from 1 to 4, are picking up objects, dropping off objects, reporting locations, and disarming mines. The robot is capable of sensing objects to pick up, targets to report, mines to disarm, whether it has an object already, and whether it is at the drop off location or is in communication range.

6.2.2 Compactness evaluation results

In Fig. 4, we show the behavior tree b_m synthesized by our method from PPDDL specification P_m. We observe that the

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 1 Marine Domain - Compactness Evaluation: For each method, given a domain and problem specification denoted by $\mathcal{S}$, we evaluate behavior tree (BT) compactness of the synthesized BT by considering the number of action nodes n_a, the number of condition nodes n_c, the total number of nodes n (scaling factor F_n with respect to the proposed PPDDLtoBT method), and the average number of active conditions for selecting the active action $\overline{N}_{c/a}$ (scaling factor $F_{c/a}$ with respect to PPDDLtoBT). For both the deterministic baseline specification D_m and the PPDDL specification P_m, we observe that our proposed method PPDDLtoBT synthesizes the most compact BT, i.e. this BT has the fewest number of nodes (1.56 x to 26.11 x fewer) and requires the fewest number of active conditions for selecting the active action on average (1.22 x to 11.24 x fewer)

Method	$\mathcal{S}$	n_a	n_c	n	F_n	$\overline{N}_{c/a}$	$F_{c/a}$
PPDDLtoBT (proposed)	D_m	7	17	44	1 x	5.70	1 x
Keeping don't cares	D_m	7	29	69	1.57 x	6.95	1.22 x
Without simplification	D_m	64	384	705	16.02 x	47	8.25 x
PPDDLtoBT (proposed)	P_m	6	10	27	1 x	4.19	1 x
Keeping don't cares	P_m	6	18	42	1.56 x	5.38	1.29 x
Without simplification	P_m	64	384	705	26.11 x	47	11.24 x

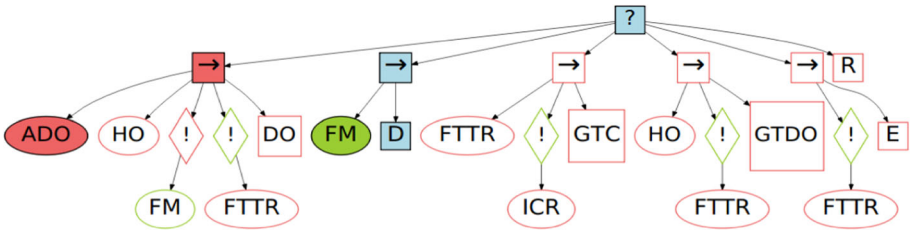


Fig. 4 The 27-node simplified behavior tree synthesized from the marine domain P_m. This tree would have 705 nodes without simplification and is equivalent in performance to the optimal policy. The conditions are *found-target-to-report* (FTTR), *has-object* (HO), *at-drop-off* (ADO), *in-comms-range* (ICR), *found-mine* (FM). The actions are *drop-off* (DO), *disarm* (D), *go-to-comms* (GTC), *go-to-drop-off* (GTDO), *explore* (E), and *report* (R). Note that the flow of node execution is from left to right, and node activity denoted by colored shading (active) or a lack thereof (inactive). Nodes with a *failure* status are red, those with a *success* status are green, and nodes that are returning a *running* status are blue (Color figure online)

subtrees in b_m collectively encompass logic for each of the actions associated with a reward. This is to be expected since this behavior tree is a simplified, but equivalent representation of the optimal policy for the corresponding marine MDP. We note that b_m has 27 nodes, in contrast with the 705-node behavior tree generated from the optimal policy before simplification.

In Table 1, we first consider compactness by examining the number of action nodes, n_a, condition nodes, n_c, and nodes altogether, n, found in the behavior trees synthesized by our proposed method, PPDDLtoBT, and the two comparison methods defined in 6.1.1. Given deterministic baseline specification D_m (Appx. A, Fig. 12), we observe that the behavior tree synthesized by PPDDLtoBT has 1) 1.57 x fewer nodes than it would without the stage of simplification the removes don't cares, and 2) 16.02 x fewer nodes than it would without any policy simplification. Given PPDDL specification P_m (Appx. A, Fig. 11), we see the same trend, but more pronounced, i.e., the PPDDLtoBT-synthesized behavior tree 1) has 1.56 x fewer nodes than the *keeping don't cares* behavior tree, and 2) 26.11 x fewer nodes than the *without simplification* behavior tree.

By examining n_a and n_c for both specifications, D_m and P_m, we see that this trend is due to fewer conditions being removed during simplification in both comparison methods, and in fact no actions or conditions being removed through simplification by definition in the *without simplification* method. We note that the number of nodes (n, n_a and n_c) in the behavior tree synthesized by the *without simplification* method is the same for both D_m and P_m. This is because these domains have the same number of states and one subtree is synthesized per state-action pair in the optimal policy. Although the number of nodes and overall structure of these behavior trees is the same, the logic is not because the domains and the policies that these behavior trees represent are different.

Next, in Table 1, we evaluate the compactness of the synthesized behavior trees by considering the average number of active conditions for selecting the active action, $\overline{N}_{c/a}$. Given D_m, the PPDDLtoBT synthesized behavior tree has 1) an $\overline{N}_{c/a}$ 1.22 x smaller than the $\overline{N}_{c/a}$ of the *keeping don't cares* behavior tree, and 2) 8.25 x smaller than the *without simplification* behavior tree $\overline{N}_{c/a}$. This trend is more apparent given P_m with 1.29 x and 11.24 x smaller respectively. This not only

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

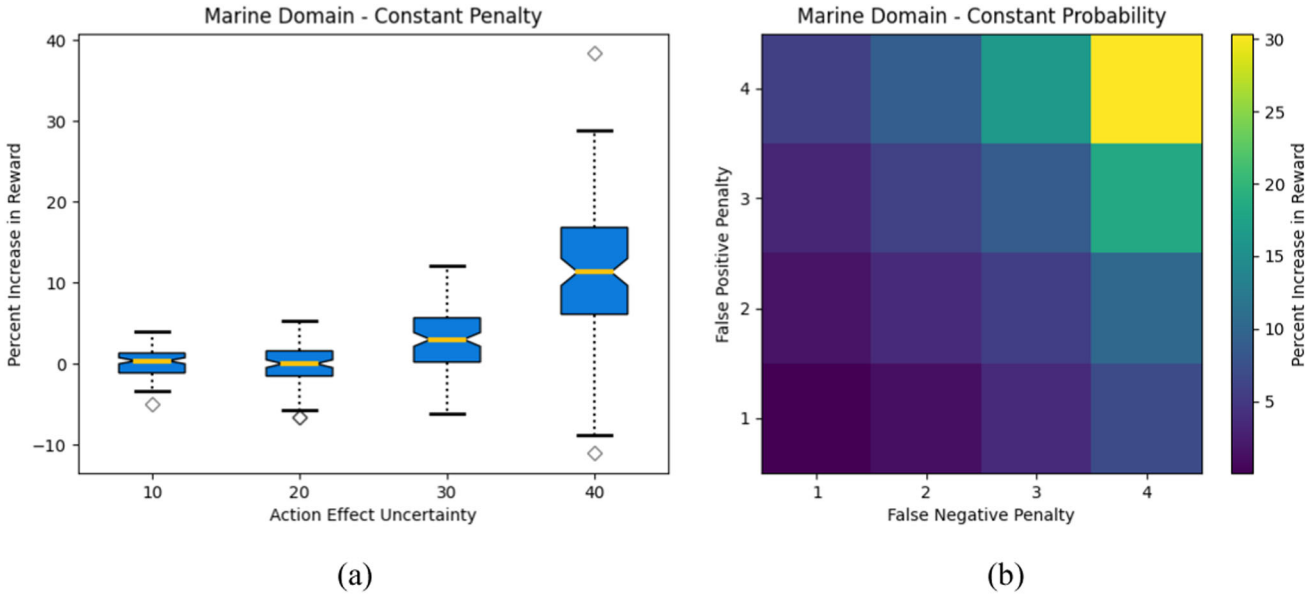


Fig. 5 Comparison of reward achieved in the marine domain over 100 trials by a behavior tree synthesized with a probabilistic specification P_m and one with a deterministic specification D_m as a baseline. In (a) we consider the domain with constant penalty magnitudes but action effect uncertainty ranging from 10-40% identically for all nondeterministic actions. (a) shows that the percent increase in reward for our method given P_m over it with D_m is always positive, with the median ranging from near 0 for $< 20\%$ action effect uncertainty and approximately 3-11% for $> 20\%$ uncertainty. In (b) we consider the domain with constant action effect uncertainty at 30% and a range of false negative and positive penalty magnitudes. (b) shows $< 1\%$ to $> 30\%$ increase in reward for our method with P_m, and the clear pattern of benefit reflects the distinct differences between action effects in the marine domain (Color figure online)

shows the benefit of our simplification method at increasing synthesized behavior tree compactness, but it also suggests greater behavior tree efficiency.

6.2.3 Probabilistic vs. deterministic evaluation results

In Fig. 5a, we consider the percent increase in average reward achieved by our method given P_m versus D_m respectively, as the action effect uncertainty is increased from 10% to 40% identically for all nondeterministic actions. A positive percent increase means our method given P_m achieved a higher average reward than our method given D_m. For clarity in this comparison, we keep penalties associated with false positive and false negative action outcomes constant. In Fig. 5a, we observe that the percent increase is always positive, becomes larger as the action effect uncertainty does, and ranges from $< 1\%$ to 11%. With $< 20\%$ action effect uncertainty, the percent increase in reward is positive, but minimal, i.e. $< 1\%$. We theorize that this is due to the actions being sufficiently distinct given their unique reward magnitudes and state effects. In other words, for lower action effect uncertainty, the problem is easy enough to solve through approximation of P_m with D_m. However, with action effect uncertainty $> 20\%$, we observe the median percent increase to range from approximately 3% to 11%. This suggests that approximation with D_m as input is insufficient, and that our method can accommodate and harness information from P_m to synthesize high-performing behavior trees for domains with higher action effect uncertainty.

In Fig. 5b, we consider the percent increase in average reward achieved by our method given P_m versus D_m again, but with constant action effect uncertainty at 30% and a range of false positive and false negative penalty magnitudes from 1 to 4. We observe that the percent increase in average reward our method achieves with P_m is always positive, ranging from $< 1\%$ to over 30% with an average of 8%. A notable gradient pattern in percent increase is apparent as false positive and false negative penalty magnitudes are increased, with the maximum benefit being correlated with the maximum penalty magnitudes. This trend demonstrates that our method is capable of synthesizing high-performing behavior trees for domains that involve actions with a broad range of magnitudes of consequence for failure.

6.2.4 Sensitivity analysis results

In Table 2, we report the reward obtained in the correct MDP by behavior trees synthesized from inaccurate PPDDL representations of the actual MDP. The PPDDL representation of the correct MDP is shown in Appendix 11, and we inject mistakes of the types described in Sec. 6.1.3. We limit one mistake per action, and inject a mistake into a single action,

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Marine Domain - Sensitivity Analysis: Given the accurate domain and problem specification for the marine domain, we inject the following types of mistakes into actions: a missing precondition (MP), an added precondition (AP), a higher reward (HR), a higher penalty (HP), a lower action efficacy likelihood (LL), and a higher action efficacy likelihood (HL). The optimal policy achieves a reward of 62.075, and we report the rewards obtained in the correct MDP by the behavior trees synthesized from the domain with varying numbers of mistakes

No mistakes: 62.075	MP	AP	HR	HP	LL	HL
One action	62.075	62.075	61.223	62.075	62.075	61.189
Half (3) actions	62.075	2.638	61.168	62.075	62.037	61.229
All (4 to 7) actions	62.075	1.911	61.229	62.075	62.075	61.244

approximately half of the actions (3 of 7), and all possible actions (4 to 7 depending on mistake type). We compare against a reward of 62.075 that is achieved by the optimal policy, i.e., the same reward achieved by the behavior tree synthesized from the accurate PPDDL specification.

First, we observe that missing preconditions and higher penalty magnitudes did not have an effect on the reward. Next, we see that higher rewards, lower likelihoods, and higher likelihoods each decreased the reward by at most 1.5%. Lastly, we see that an added precondition to one action has no effect. However, an added precondition to more than one action reduced the reward by 96% given 3 actions affected, and 97% given 7 actions affected. Overall, these results demonstrate that our method is robust to most types and numbers of mistakes in the marine domain, but is affected more significantly when more than one incorrect precondition is introduced.

6.2.5 Policy and behavior tree equivalence evaluation results

Given 20 variations of the marine domain in PPDDL, differing in action effect uncertainty or penalty magnitudes (as described in Sec. 6.1.2), we evaluate the optimal policy and the synthesized behavior tree independently over 1000 trials. Given that our method is constructed such that the behavior tree is synthesized from and should be logically equivalent to the optimal policy, we expect that both the policy and the behavior tree should trigger the same actions in every state and always achieve the same reward as a result. This is exactly what we see in every variation of the marine domain, i.e., the optimal policy and synthesized behavior tree perform identically, and so they exclusively achieve the same reward. These identical rewards range from -96.4 to 98.2, depending on the domain variation, with an average reward of 1.74.

6.2.6 Runtime report results

In the same 20 variations of the marine domain we consider for the equivalence evaluation above in Sec. 6.2.5, we examine the time it takes our method to synthesize the final behavior tree to demonstrate consistency and feasibility of use. In Fig. 6, we see that the runtime ranges from 0.27 to 0.40 seconds, with a mean of 0.30 seconds and a standard deviation of 0.04 seconds.

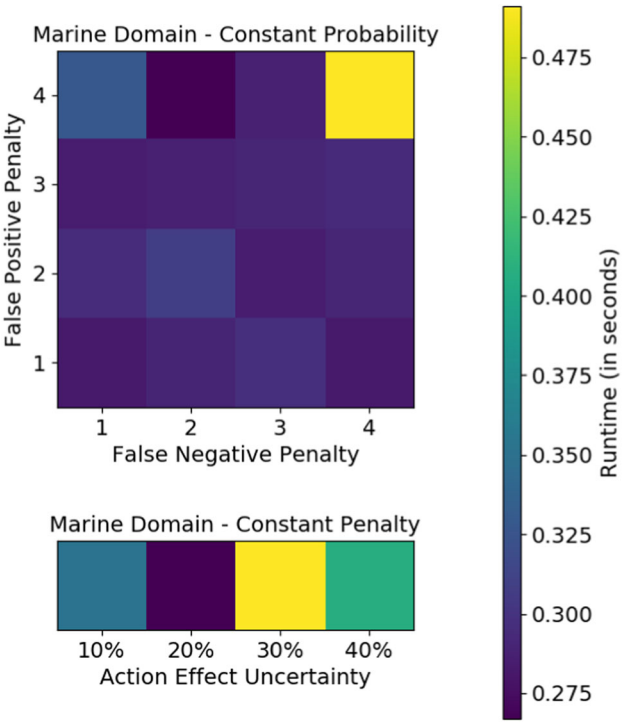


Fig. 6 For 20 variations of the marine domain, we show the time it takes our method to synthesize a behavior tree from PPDDL input. We discuss these results in Sec. 6.2.6 (Color figure online)

6.3 Intervention for infant mobility

6.3.1 Experimental setup

Next, we present an intervention for infant mobility scenario depicted in Fig. 7. The goal in this domain is to encourage movement in children due to the associated developmental benefits, particularly for children experiencing developmental delays and motor disabilities (Helmi et al., 2022). The robot in this scenario can incrementally move toward or away from the infant, engage in a keep away from the infant game,

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 3 Infant Domain - Compactness Evaluation: For each method, given a domain and problem specification denoted by $\mathcal{S}$, we evaluate behavior tree compactness of the synthesized behavior tree by considering the number of action nodes n_a, the number of condition nodes n_c, the total number of nodes n (scaling factor F_n with respect to the proposed PPDDLtoBT method), and the average number of active conditions for selecting the active action $\overline{N}_{c/a}$ (scaling factor $F_{c/a}$ with respect to PPDDLtoBT). For both the deterministic baseline specification D_i and the PPDDL specification P_i, we observe that our proposed method PPDDLtoBT synthesizes the most compact BT, i.e. this BT has the fewest number of nodes (4.51 x to 25.64 x fewer) and requires the fewest number of active conditions for selecting the active action on average (3.04 x to 12.47 x fewer)

Method	$\mathcal{S}$	n_a	n_c	n	F_n	$\overline{N}_{c/a}$	$F_{c/a}$
PPDDLtoBT (proposed)	D_i	7	14	39	1 x	5.52	1 x
Keeping don't cares	D_i	7	188	360	9.23 x	20.46	3.7 x
Without simplification	D_i	54	540	1000	25.64 x	68.83	12.47 x
PPDDLtoBT (proposed)	P_i	8	26	69	1 x	8	1 x
Keeping don't cares	P_i	8	204	380	4.51 x	24.28	3.04 x
Without simplification	P_i	54	540	1000	14.49 x	68.80	8.6 x



Fig. 7 Our proposed method is directly applicable to the above shown infant mobility domain (Helmi et al., 2022). *Left* is the assistive robot, which can blow bubbles, play sounds, flash lights, and move playfully with the intent of encouraging developmentally beneficial movement in children. *Right* shows the robot interacting with children in a play area (Color figure online)

fully go to the infant's location, spin around, play sounds, flash lights, blow bubbles, and refill bubble solution. All actions except refilling bubble solution are associated with a reward (of often identical magnitude), and cause the infant to move toward the robot when effective (identical action effect).

Actions differ mainly in preconditions, i.e. the states in which action execution can occur. This presents a different kind of challenge than the marine domain because of greater reward magnitude uniformity and action effect similarity. In combination, these traits make the infant domain more ambiguous. Formally, the goal is to maximize the reward, which represents the amount of time that the infant is moving toward the robot.

The robot is capable of sensing whether the infant is stationary or moving toward/away, facing toward/away, whether the infant's view of the robot is obstructed, whether the robot has bubble solution, and how far away the infant is as denoted by direct social interaction (closest), social interaction (middle), and solitary play (furthest).

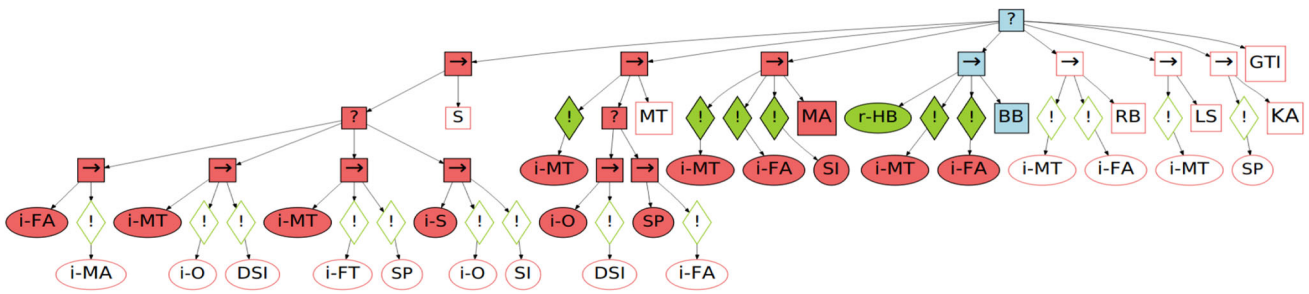


Fig. 8 The 69-node simplified behavior tree synthesized from the infant mobility PPDDL specification P_i. This tree would have 1000 nodes without simplification and is equivalent in performance to the optimal policy. The conditions are *infant-facing-toward* (i-FT), *infant-facing-away* (i-FA), *infant-view-obstructed* (i-O), *direct-social-interaction* (DSI), *social-interaction* (SI), *solitary-play* (SP), *infant-moving-toward* (i-MT), *infant-moving-away* (i-MA), *infant-stationary* (i-S), and *robot-has-bubbles* (r-HB). The actions are *spin* (S), *move-toward* (MT), *move-away* (MA), *blow-bubbles* (BB), *refill-bubbles* (RB), *lights-and-sounds* (LS), *keep-away* (KA), and *go-to-infant* (GTI) (Color figure online)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

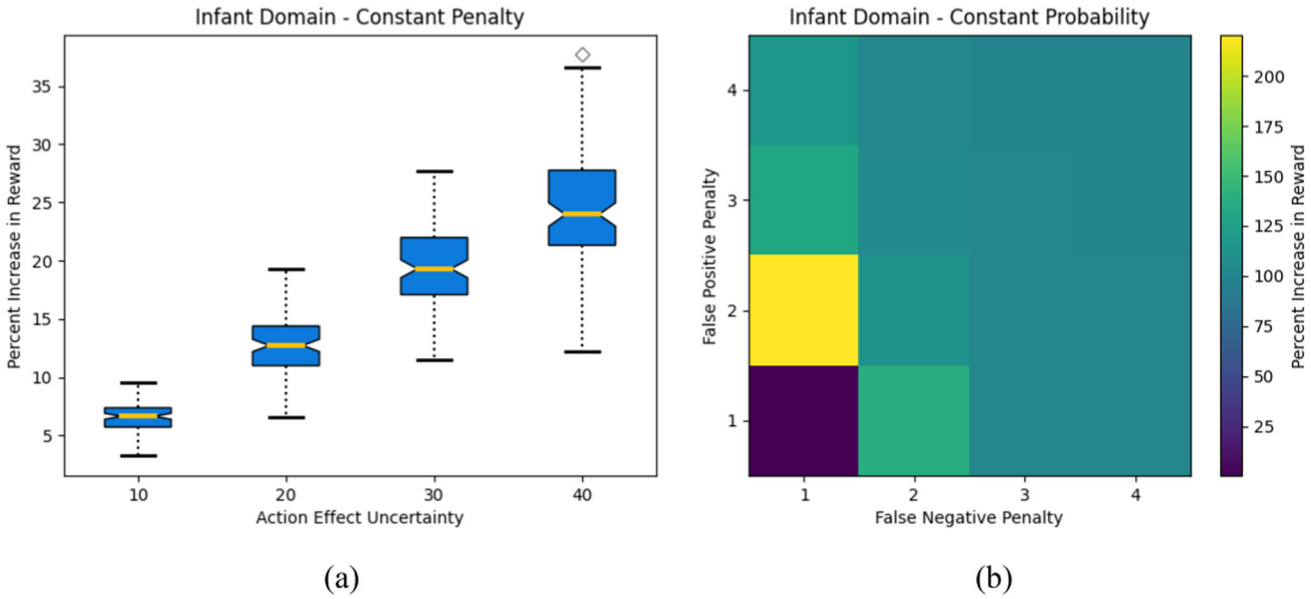


Fig. 9 Comparison of reward achieved in the infant mobility domain over 100 trials by a behavior tree synthesized with a probabilistic specification P_i and one with a deterministic specification D_i as a baseline. In (a) we consider the domain with constant penalty magnitudes but action effect uncertainty ranging from 10-40% identically for all nondeterministic actions. (a) shows approximately 7-24% increase in reward for our method given P_i over our method with D_i. In (b) we consider the domain with constant action effect uncertainty at 30% and a range of false negative and positive penalty magnitudes. (b) shows $<$ 1%-220% increase in reward for our method given P_i, and the pattern of benefit reflects ambiguity in the infant domain due to limited action effect differentiability (Color figure online)

6.3.2 Compactness evaluation results

In Fig. 8, we show a behavior tree b_i synthesized by our method given PPDDL specification P_i. We observe that not all actions that can result in a reward are present in b_i. Such a presence is not necessary because infant domain actions often result in the same state effect, i.e. the infant moving toward the robot. We also note that in contrast with the 1000-node behavior tree generated from the optimal policy before the Alg. 3 simplification, b_i has only 69 nodes while maintaining equivalent functionality.

In Table 3, we first evaluate compactness of the behavior trees synthesized by our proposed method, PPDDLtoBT, and the two comparison methods by examining the number of action nodes, n_a, condition nodes, n_c, and nodes altogether, n, in each tree. Given deterministic baseline specification D_i, our proposed PPDDLtoBT method synthesized a behavior tree with 1) 9.23 x fewer nodes than it would if don't cares were not removed during simplification, and 2) 25.64 x fewer nodes than it would if the simplification stage of PPDDLtoBT were skipped entirely. Given PPDDL specification P_i, we see the same trend where the PPDDLtoBT-synthesized behavior tree has 1) 4.51 x fewer nodes than the *keeping don't cares* behavior tree, and 2) 14.49 x fewer nodes than the *without simplification* behavior tree.

For both specifications, D_i and P_i, we observe that the PPDDLtoBT method behavior tree is the most compact because both the *without simplification* method and the *keeping don't cares* method fail at sufficiently reducing n_c. Additionally, whereas the *keeping don't cares* method successfully minimizes n_a, the *without simplification* methods does not at all. We note that although the node numbers (n_a, n_c, and n) are the same for the *without simplification* method given either specification, this is just because the policies these trees represent have the same number of state-action pairs and is not an indication of equivalent functionality. These compactness results match what we observed for the marine domain (see Sec. 6.2). The differences in detail within the similar compactness trends we see for the infant and marine domains (e.g., the differing range in scaling factor F_n between methods) are due to how much redundancy, i.e., potential for simplification, there happens to be in the optimal policy for each given domain.

In Table 3, we also consider synthesized behavior tree compactness by examining the average number of active conditions for selecting the active action, $\overline{N}_{c/a}$. Given D_i, the PPDDLtoBT synthesized behavior tree has 1) an $\overline{N}_{c/a}$ 3.7 x smaller than the $\overline{N}_{c/a}$ of the *keeping don't cares* behavior tree, and 2) 12.47 x smaller than the *without simplification* behavior tree $\overline{N}_{c/a}$. Similarly, given P_i, this trend holds with the PPDDLtoBT behavior tree $\overline{N}_{c/a}$ being 3.04 x and 8.6 x smaller respectively. These results match what we saw for the marine domain; they show our proposed method's efficacy

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 4 Infant Domain - Sensitivity Analysis: Given the accurate domain and problem specification for the infant domain, we inject the following types of mistakes into actions: a missing precondition (MP), an added precondition (AP), a higher reward (HR), a higher penalty (HP), a lower action efficacy likelihood (LL), and a higher action efficacy likelihood (HL). The optimal policy achieves a reward of 0.821, and we report the rewards obtained in the correct MDP by the behavior trees synthesized from the domains with varying numbers of mistakes

No mistakes: 0.821	MP	AP	HR	HP	LL	HL
One action	0.724	0.821	0.821	0.821	0.821	0.821
Half (4) actions	0.664	0.681	-3.044	0.803	0.803	-3.001
All (7 to 9) actions	0.492	0.505	-2.968	0.735	0.735	-2.925

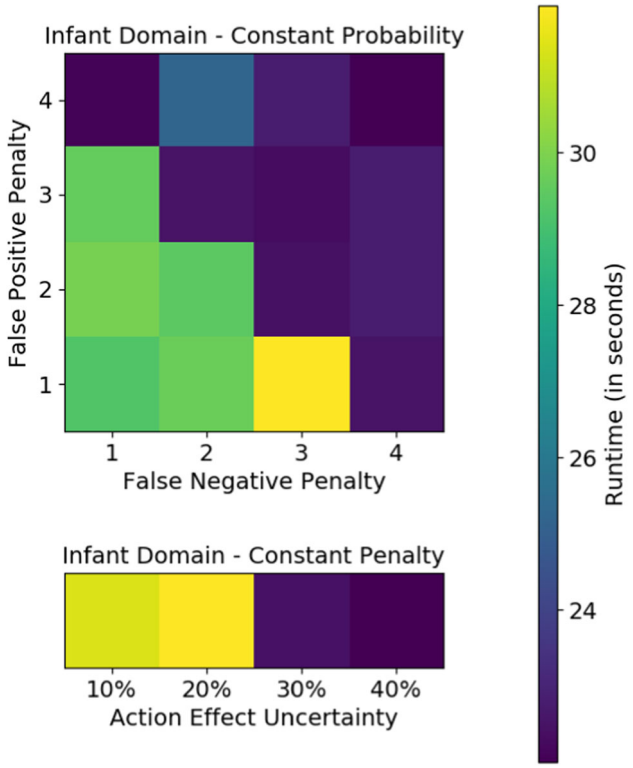


Fig. 10 For 20 variations of the infant domain, we show the time our method takes to synthesize a behavior tree from PPDDL. We discuss these results in Sec. 6.3.6 (Color figure online)

at increasing synthesized behavior tree compactness and the likelihood behavior tree efficiency.

6.3.3 Probabilistic vs. deterministic evaluation results

In Fig. 9a, we consider the percent increase in average reward achieved our method given the full probabilistic infant domain specification, P_i, versus the deterministic version, D_i, as the action effect uncertainty is increased from 10% to 40% identically for all nondeterministic actions. For clarity, we keep the false positive and false negative penalty magnitudes constant. We observe that the percent increase in average reward ranges from 7% to 24%, increasing as action effect uncertainty increases. This trend demonstrates that our method's ability to take probabilistic input such as P_i results in a higher reward that increases proportionally with the likelihood of action success. This is because the infant domain is sufficiently complex that D_i is not a viable approximation of P_i for any action effect uncertainty in the given range. We also observe that the percent increase in average reward for the infant domain exceeds that which our method achieved with the marine domain. This suggests that our method is well-suited for domains with higher degrees of ambiguity, i.e., not immediately apparent differentiability between actions both in terms of preconditions and effects.

In Fig. 9b, we consider the percent increase in average reward achieved by our method given P_i versus D_i again, but with constant action effect uncertainty of 30% and a range of false positive and false negative penalty magnitudes from 1 to 4. We observe that the percent increase ranges from $< 1\%$ to over 220% with an average of 108%. These results show that our method given P_i exclusively outperformed that given D_i, further demonstrating the benefit associated with our method's ability to take probabilistic input. Note that we do not see the gradient pattern of increase that we observed for the marine domain in the same experiment. This is not a weakness, but rather demonstrates the difference between the types of domains that the marine and infant domain are respectively. Recall, the infant domain actions are more ambiguous than marine domain actions, i.e. infant domain actions often have identical reward magnitudes and affect the world state in the same way.

6.3.4 Sensitivity analysis results

In Table 4, we report the reward obtained in the correct MDP by behavior trees synthesized from inaccurate PPDDL representations of the actual MDP. The PPDDL representation of the correct MDP is shown in Appendix 14, and we inject mistakes of the types described in Sec. 6.1.3. We limit one mistake per action, and inject a mistake into a single action, approximately half of the actions (4 of 9), and all possible actions (7 to 9 depending on mistake type). The behavior tree synthesized from the accurate PPDDL specification (that is equivalent functionally to the optimal policy) achieves a reward of 0.821, which we compare against in this analysis.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

First, we observe that a mistake in a single action only has an effect for the missing precondition case with a 12% reward decrease. Next, we see that missing or added preconditions (AP), higher penalties (HP), and lower likelihoods (LL) result in a gradually decreasing reward (between 2% and 40% lower) with increasing numbers of mistakes. Lastly, we see a more drastic decrease in reward given the injection of higher reward (HR) or likelihoods (HL) mistakes. For these mistake types, we see no effect for single mistakes, and an average of a 464% decrease in reward given mistakes are present in between 4 and 9 actions. Overall, these results demonstrate that in the infant domain our method 1) is robust to single mistakes regardless of type, 2) gradually degrades in performance for the majority of mistake types, and 3) is most susceptible to mistakes that represent actions as more effective than they actually are (i.e., higher rewards for or likelihoods of action efficacy).

6.3.5 Policy and behavior tree equivalence evaluation results

Given 20 variations of the infant domain in PPDDL, differing in action effect uncertainty or penalty magnitudes (as described in Sec. 6.1.2), we compare the reward achieved by the optimal policy and the reward achieved by synthesized behavior tree over 1000 trials. As we expect, the rewards are always equivalent in every infant domain variation because the same actions are triggered by the optimal policy and the synthesized behavior tree in every state. Just as we saw in the marine domain, this result in the infant domain empirically supports our claim that we maintain the logic of the optimal policy in the behavior tree our method synthesizes from it. The identically achieved rewards range from 0.53 to 2.38, depending on the domain variation, with an average reward of 1.27.

6.3.6 Runtime report results

In the same 20 variations of the infant domain we consider for the equivalence evaluation above in Sec. 6.3.5, we examine the time it takes our method to synthesize the final behavior tree to demonstrate consistency and feasibility of use. In Fig. 10, we see that the runtime ranges from 17 to 32 seconds, with a mean of 24 seconds and a standard deviation of 4 seconds.

7 Conclusion and future work

We presented an algorithm that synthesizes a behavior tree with equivalent performance to the optimal policy in the MDP that the initially inputted PPDDL specification represents. Our results demonstrate that our algorithm produced behavior trees with approximately between 15x and 26x fewer nodes and an average of between 8x and 13x fewer active conditions for selecting the active action than they would without simplification. This maximized compactness and execution efficiency suggests a higher degree of user interpretability and benefit over other potential MDP policy representations. We also demonstrated that our method is capable of handling domains with distinct or ambiguous actions, providing an increasingly higher average reward as domains become more challenging with respect to greater action effect uncertainty or action failure penalty magnitudes.

Our sensitivity analysis demonstrated that our method is still capable of synthesizing a behavior tree when a variety of types and numbers of mistakes are present in the input PPDDL specification, and that for the majority of cases this only resulted in graceful reward degradation. Finally, we empirically confirmed that the synthesized behavior trees do perform equivalently to the optimal policy, and that this synthesis consistently takes at most 32 seconds for the most challenging domain we examined.

In the future, it would be interesting to extend this method to accommodate more complex domains that involve heterogeneous multi-robot teams or higher degrees of current state uncertainty or partial observability. An example avenue might entail additional behavior tree simplification and optimization, or even the extension of certain node functionality.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

A Domain and problem specifications

```
(define (domain marine)
        (:requirements :equality :typing :mdp)
        (:types None)
        (:predicates
                (has-object ?x - None)
                (found-object ?x - None)
                (found-mine ?x - None)
                (at-drop-off ?x - None)
                (found-target-to-report ?x - None)
                (in-comms-range ?x - None))
        (:action pick-up
                :parameters (?x - None)
                :precondition (and (found-object ?x) (not (has-object ?x)))
                :effect (probabilistic 0.7 (and (has-object ?x) (not (found-object ?x)) (increase reward 1))
                        0.15 (and (not (has-object ?x)) (not (found-object ?x)) (increase reward -1))
                        0.15 (increase reward -1)))
        (:action drop-off
                :parameters (?x - None)
                :precondition (and (has-object ?x) (at-drop-off ?x))
                :effect (probabilistic 0.7 (and (not (has-object ?x)) (increase reward 2))
                        0.15 (and (has-object ?x) (increase reward -2))
                        0.15 (and (not (at-drop-off ?x)) (increase reward -2))))
        (:action disarm
                :parameters (?x - None)
                :precondition (found-mine ?x)
                :effect (probabilistic 0.9 (and (increase reward 4) (not (found-mine ?x)))
                        0.05 (and (found-mine ?x) (increase reward -4))
                        0.05 (increase reward -4)))
        (:action go-to-comms
                :parameters (?x - None)
                :precondition (and (not (in-comms-range ?x)) (found-target-to-report ?x))
                :effect (in-comms-range ?x))
        (:action go-to-drop-off
                :parameters (?x - None)
                :precondition (and (has-object ?x) (not (at-drop-off ?x)))
                :effect (at-drop-off ?x))
        (:action report
                :parameters (?x - None)
                :precondition (and (found-target-to-report ?x) (in-comms-range ?x))
                :effect (probabilistic 0.8 (and (not (found-target-to-report ?x)) (increase reward 3))
                        0.1 (and (found-target-to-report ?x) (increase reward -3))
                        0.1 (and (not (in-comms-range ?x)) (increase reward -3))))
        (:action explore
                :parameters (?x - None)
                :precondition (and (not (has-object ?x)) (not (found-mine ?x)) (not (found-target-to-report ?x)))
                :effect (probabilistic 0.2 (found-target-to-report ?x)
                        0.15 (found-object ?x)
                        0.1 (found-mine ?x)
                        0.55 (has-object ?x)))
)
```

Fig. 11 P_m: Probabilistic Marine Domain Specification (PPDDL)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

```
(define (domain marine)
        (:requirements :equality :typing :probabilistic-effects :rewards)
        (:types None)
        (:predicates
                (has-object ?x - None)
                (found-object ?x - None)
                (found-mine ?x - None)
                (at-drop-off ?x - None)
                (found-target-to-report ?x - None)
                (in-comms-range ?x - None))
        (:action pick-up
                :parameters (?x - None)
                :precondition (and (found-object ?x) (not (has-object ?x)))
                :effect (and (has-object ?x) (not (found-object ?x)) (increase reward 1)))
        (:action drop-off
                :parameters (?x - None)
                :precondition (and (has-object ?x) (at-drop-off ?x))
                :effect (and (not (has-object ?x)) (increase reward 2)))
        (:action disarm
                :parameters (?x - None)
                :precondition (found-mine ?x)
                :effect (and (increase reward 4) (not (found-mine ?x))))
        (:action go-to-comms
                :parameters (?x - None)
                :precondition (and (not (in-comms-range ?x)) (found-target-to-report ?x))
                :effect (in-comms-range ?x))
        (:action go-to-drop-off
                :parameters (?x - None)
                :precondition (and (has-object ?x) (not (at-drop-off ?x)))
                :effect (at-drop-off ?x))
        (:action report
                :parameters (?x - None)
                :precondition (and (found-target-to-report ?x) (in-comms-range ?x))
                :effect (and (not (found-target-to-report ?x)) (increase reward 3)))
        (:action explore
                :parameters (?x - None)
                :precondition (and (not (has-object ?x)) (not (found-mine ?x)) (not (found-target-to-report ?x)))
                :effect (probabilistic 0.2 (found-target-to-report ?x)
                        0.15 (found-object ?x)
                        0.1 (found-mine ?x) ;what about mine-live?
                        0.55 (has-object ?x)))
)
```

Fig. 12 D_m: Deterministic Marine Domain Specification

```
(define (problem marine)
        (:domain marine)
        (:objects X - None )
        (:init (has-object X)
                (found-object X)
                (found-mine X)
                (found-target-to-report X)
                (at-drop-off X)
                (in-comms-range X))
        (:goal (and (has-object)))
)
```

Fig. 13 Marine Problem Specification

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

```
(define (domain infant-mobility)
    (:requirements :equality :typing :mdp)
    (:types None activity distance orientation)
    (:predicates
        (has-bubbles ?x - None)
        (infant-activity ?x - activity)
        (infant-robot-distance ?x - distance)
        (infant-orientation ?x - orientation))
    (:constants
        direct-social-interaction social-interaction solitary-play - distance
        moving-toward moving-away stationary - activity
        toward away obstructed - orientation)
    (:constraints
        (and (forall (?act - activity) (at-most-once (infant-activity ?act)))
            (forall (?dst - distance) (at-most-once (infant-robot-distance ?dst)))
            (forall (?otn - orientation) (at-most-once (infant-orientation ?otn)))))
    (:action spin
        :parameters ()
        :precondition (and (infant-activity stationary) (not (infant-orientation obstructed)))
        :effect (probabilistic 0.7 (and (infant-activity moving-toward) (increase reward 2))
            0.15 (increase reward -2)
            0.15 (increase reward -2)))
    (:action move-toward
        :parameters ()
        :precondition (not (infant-robot-distance direct-social-interaction))
        :effect (probabilistic 0.7 (and (infant-activity moving-toward) (increase reward 2))
            0.15 (increase reward -2)
            0.15 (increase reward -2)))
    (:action move-away
        :parameters ()
        :precondition (or (infant-robot-distance direct-social-interaction)
                (infant-robot-distance social-interaction)
                (infant-robot-distance solitary-play))
        :effect (probabilistic 0.7 (and (infant-activity moving-toward) (increase reward 2))
            0.15 (increase reward -2)
            0.15 (increase reward -2)))
    (:action blow-bubbles
        :parameters (?x - None)
        :precondition (and (infant-robot-distance social-interaction) (has-bubbles ?x) (infant-orientation toward))
        :effect (probabilistic 0.9 (and (infant-activity moving-toward) (increase reward 3))
            0.05 (increase reward -3)
            0.05 (and (not (has-bubbles ?x)) (increase reward -3))))
    (:action refill-bubbles
        :parameters (?x - None)
        :precondition (not (has-bubbles ?x))
        :effect (and (has-bubbles ?x)))
    (:action play-sounds
        :parameters ()
        :precondition (and (not (infant-activity moving-toward)) (infant-orientation away))
        :effect (probabilistic 0.7 (and (infant-activity moving-toward) (increase reward 2))
            0.15 (increase reward -2)
            0.15 (increase reward -2)))
    (:action lights-and-sounds
        :parameters ()
        :precondition (and (infant-activity moving-away) (infant-orientation away))
        :effect (probabilistic 0.8 (and (infant-activity moving-toward) (increase reward 2))
            0.1 (increase reward -2)
            0.1 (increase reward -2)))
    (:action keep-away
        :parameters ()
        :precondition (and (infant-robot-distance direct-social-interaction) (infant-orientation toward))
        :effect (probabilistic 0.9 (and (infant-robot-distance social-interaction)
                    (infant-activity moving-toward) (increase reward 2))
            0.05 (and (infant-robot-distance social-interaction) (infant-activity stationary) (increase reward -2))
            0.05 (and (infant-robot-distance solitary-play) (infant-activity stationary) (increase reward -2))))
    (:action go-to-infant
        :parameters ()
        :precondition (and (infant-robot-distance solitary-play) (infant-orientation obstructed))
        :effect (probabilistic 0.45 (and (infant-robot-distance direct-social-interaction) (infant-activity moving-toward)
                    (infant-orientation toward) (increase reward 2))
            0.45 (and (infant-robot-distance direct-social-interaction) (infant-activity moving-toward)
                (infant-orientation away) (increase reward 2))
            0.05 (and (infant-robot-distance direct-social-interaction) (infant-activity moving-away)
                (increase reward -2))
            0.05 (and (infant-robot-distance social-interaction) (infant-activity moving-away)
                (increase reward -2))))
)
```

Fig. 14 P_i: Probabilistic Infant Domain Specification (PPDDL)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

```
(define (domain infant-mobility)
        (:requirements :equality :typing :rewards)
        (:types None activity distance orientation)
        (:predicates
                (has-bubbles ?x - None)
                (infant-activity ?x - activity)
                (infant-robot-distance ?x - distance)
                (infant-orientation ?x - orientation))
        (:constants
                direct-social-interaction social-interaction solitary-play - distance
                moving-toward moving-away stationary - activity
                toward away obstructed - orientation)
        (:constraints
                (and (forall (?act - activity) (at-most-once (infant-activity ?act)))
                (forall (?dst - distance) (at-most-once (infant-robot-distance ?dst)))
                (forall (?otn - orientation) (at-most-once (infant-orientation ?otn)))))
        (:action spin
                :parameters ()
                :precondition (and (infant-activity stationary) (not (infant-orientation obstructed)))
                :effect (and (infant-activity moving-toward) (increase reward 2)))
        (:action move-toward
                :parameters ()
                :precondition (not (infant-robot-distance direct-social-interaction))
                :effect (and (infant-activity moving-toward) (increase reward 2)))
        (:action move-away
                :parameters ()
                :precondition (or (infant-robot-distance direct-social-interaction)
                                  (infant-robot-distance social-interaction)
                                  (infant-robot-distance solitary-play))
                :effect (and (infant-activity moving-toward) (increase reward 2)))
        (:action blow-bubbles
                :parameters (?x - None)
                :precondition (and (infant-robot-distance social-interaction) (has-bubbles ?x) (infant-orientation toward))
                :effect (and (infant-activity moving-toward) (increase reward 3)))
        (:action refill-bubbles
                :parameters (?x - None)
                :precondition (not (has-bubbles ?x))
                :effect (and (has-bubbles ?x)))
        (:action play-sounds
                :parameters ()
                :precondition (and (not (infant-activity moving-toward)) (infant-orientation away))
                :effect (and (infant-activity moving-toward) (increase reward 2)))
        (:action lights-and-sounds
                :parameters ()
                :precondition (and (infant-activity moving-away) (infant-orientation away))
                :effect (and (infant-activity moving-toward) (increase reward 2)))
        (:action keep-away
                :parameters ()
                :precondition (and (infant-robot-distance direct-social-interaction) (infant-orientation toward))
                :effect (and (infant-robot-distance social-interaction) (infant-activity moving-toward)
                                  (increase reward 2)))
        (:action go-to-infant
                :parameters ()
                :precondition (and (infant-robot-distance solitary-play) (infant-orientation obstructed))
                :effect (and (infant-robot-distance direct-social-interaction) (infant-activity moving-toward)
                                  (infant-orientation toward) (increase reward 2)))
)
```

Fig. 15 D_i: Deterministic Infant Domain Specification

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Fig. 16 Infant Problem Specification

```
(define (problem infant-mobility-problem)
        (:domain infant-mobility)
        (:objects
                direct-social-interaction social-interaction solitary-play - distance
                moving-toward moving-away stationary - activity
                toward away obstructed - orientation
                X - None)
        (:init (has-bubbles X)
                (infant-robot-distance solitary-play)
                (infant-activity moving-away)
                (infant-orientation away))
        (:goal (and (infant-activity moving-toward)))
)
```

Acknowledgements Non applicable

Author Contributions E.S. was the primary contributor in the development of the idea, algorithm implementation, and manuscript writing. G.B. helped provide the initial idea, guided the research through regular meetings with the co-authors, and contributed to the editing of the manuscript. G.A.H. provided oversight and research guidance throughout, and also contributed to the editing of the manuscript. All authors reviewed and approved the final manuscript.

Funding This work is funded in part by NSF awards 2103817 and 2024950. Graeme Best's contribution is funded in part by the Centre for Advanced Defence Research in Robotics and Autonomous Systems, Australia.

Data Availibility Statement Non applicable

Declarations

Ethical statements Non applicable

References

Constructions Aeronautiques, Adele Howe, Craig Knoblock, ISI Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, David Wilkins SRI, Anthony Barrett, Dave Christianson, et al. PDDL| the planning domain definition language. *Technical Report, Tech. Rep.*, (1998).

Ao, J., Wu, F., Wu, Y., Swikir, A., & Haddadin, S. (2024) Llm as bt-planner: Leveraging llms for behavior tree generation in robot task planning. arXiv preprint arXiv:2409.10444.

Axelsson, N., & Skantze, G. (2019) Modelling adaptive presentations in human-robot interaction using behaviour trees. In *Proc. Annual SIGdial Meeting on Discourse and Dialogue*, pages 345–352.

Banerjee, B. (2018). Autonomous acquisition of behavior trees for robot control. In *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, pages 3460–3467.

Best, G., Garg, R., Keller, J., Hollinger, G.A., & Scherer, S. (2022). Resilient multi-sensor exploration of multifarious environments with a team of aerial robots. In *Proc. Robotics: Science and Systems (RSS)*, New York City, NY, USA.

Bhat, S., Torroba, I., Özkahraman, Ö., Bore, N., Sprague, C., Xie, Y., Stenius, I., Severholt, J., Ljung, C., Folkesson, J., et al. (2020). *A cyber-physical system for hydrobatic AUVs: System integration and field demonstration*. St. Johns, NL, Canada: In Proc. IEEE OES Autonomous Underwater Vehicles Symp.

Bueno, T.P. (2017). pypddl-parser. https://github.com/thiagopbueno/pypddl-parser. Accessed: 2024-10-02.

Colledanchise, M., & Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. Taylor & Francis.

Colledanchise, M., Parasuraman, R., & Ögren, P. (2018). Learning of behavior trees for autonomous agents. *IEEE Trans. on Games, 11*(2), 183–189.

Dey, R., & Child, C. (2013) QL-BT: Enhancing behaviour tree design and implementation with Q-learning. In *Proc. IEEE Conf. on Computational Intelligence in Games (CIG)*, Niagara Falls, ON, Canada.

Fitter, N. T., Funke, R., Pulido, J. C., Eisenman, L. E., Deng, W., Rosales, M. R., Bradley, N. S., Sargent, B., Smith, B. A., & Mataric, M. J. (2019). Socially assistive infant-robot interaction: Using robots to encourage infant leg-motion training. *IEEE Robotics & Automation Magazine, 26*(2), 12–23.

Gao, C., Zhai, Y., Wang, B., & Chen, B.M. (2021). Synthesis and online re-planning framework for time-constrained behavior tree. In *Proc. IEEE Int. Conf. on Robotics and Biomimetics (ROBIO)*, pages 1896–1901.

Gugliermo, S., Schaffernicht, E., Koniaris, C., & Pecora, F. (2023). Learning behavior trees from planning experts using decision tree and logic factorization. *IEEE Robotics and Automation Letters*.

Gustavsson, O., Iovino, M., Styrud, J., & Smith, C. (2022). Combining context awareness and planning to learn behavior trees from demonstration. In *2022 31st IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, pages 1153–1160. IEEE.

Helmi, A., Noregaard, S., Giulietti, N., Logan, S.W., & Fitter, N.T. (2022). Let them have bubbles! Filling gaps in toy-like behaviors for child-robot interaction. In *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, pages 7417–7422.

Iovino, M., Scukins, E., Styrud, J., Ögren, P., & Smith, C. (2022). A survey of behavior trees in robotics and ai. *Robotics and Autonomous Systems, 154*, 104096.

Jamal, M., & Panov, A. (2021). Adaptive maneuver planning for autonomous vehicles using behavior tree on Apollo platform. In *Proc. Int. Conf. on Innovative Techniques and Applications of Artificial Intelligence*, pages 327–340. Springer.

Kartašev, M., Saler, J., & Ögren, P. (2021). Improving the performance of backward chained behavior trees that use reinforcement learning. arXiv preprint arXiv:2112.13744.

Klöckner, A. (2013). Interfacing behavior trees with the world using description logic. In *Proc. AIAA Guidance, Navigation, and Control (GNC) Conf.*, page 4636.

Lesner, B., & Zanuttini, B. (2011). *Efficient policy construction for MDPs represented in probabilistic PDDL*. Freiburg, Germany: In Proc. Int. Conf. on Automated Planning and Scheduling.

Lim, C.-U., Baumgarten, R., & Colton, S. (2010). Evolving behaviour trees for the commercial game DEFCON. In *Proc. European Conf. on the Applications of Evolutionary Computation*, pages 100–110.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Ma, X., Zhu, H., Zhu, X., & Yao, W. (2023). Evolving behavior trees for self-adaptive source searching in indoor environments. In *2023 IEEE International Conference on Unmanned Systems (ICUS)*, pages 1003–1008. IEEE.

Martín, F., Morelli, M., Espinoza, H., Lera, F.J.R., & Matellán, V. (2021). Optimized execution of PDDL plans using behavior trees. arXiv preprint arXiv:2101.01964.

Molina-Molina, J. C., Salhaoui, M., Guerrero-González, A., & Arioua, M. (2021). Autonomous marine robot based on AI recognition for permanent surveillance in marine protected areas. *Sensors, 21*(8), 2664.

Neupane, A., & Goodrich, M. (2019). Learning swarm behaviors using grammatical evolution and behavior trees. In *Proc. Int. Joint Conf. on Artificial Intelligence (IJCAI)*, pages 513–520.

Ögren, P. (2012). *Increasing modularity of UAV control systems using computer game behavior trees* (p. 4458). In Proc: AIAA Guidance, Navigation, and Control Conf.

Perez, D., Nicolau, M., O'Neill, M., & Brabazon, A. (2011). Evolving behaviour trees for the Mario AI competition using grammatical evolution. In *Proc. European Conf. on the Applications of Evolutionary Computation*, pages 123–132.

Quine, W. V. (1952). The problem of simplifying truth functions. *The American mathematical monthly, 59*(8), 521–531.

Robertson, G., & Watson, I. (2015). Building behavior trees from observations in real-time strategy games. In *2015 International symposium on innovations in intelligent systems and applications (INISTA)*, pages 1–7. IEEE.

Russell, S.J., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*. Prentice Hall, Upper Saddle River, NJ, 3rd edition.

Safronov, E., Colledanchise, M., & Natale, L. (2020). Task planning with belief behavior trees. In *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, pages 6870–6877.

Scheide, E., Best, G., & Hollinger, G.A. (2021). Behavior tree learning for robotic task planning through Monte Carlo DAG search over a formal grammar. In *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, pages 4837–4843.

Scheper, K. Y. W., Tijmons, S., de Visser, C. C., & de Croon, G. C. H. E. (2016). Behavior trees for evolutionary robotics. *Artificial life, 22*(1), 23–48.

Sprague, C. I., Özkahraman, Ö., Munafo, A., Marlow, R., Phillips, A., & Ögren, P. (2018). *Improving the modularity of AUV control systems using behaviour trees*. IEEE/OES Autonomous Underwater Vehicle Workshop (AUV), Porto, Portugal: In Proc.

Styrud, J., Iovino, M., Norrlöf, M., Björkman, M., & Smith, C. (2022). Combining planning and learning of behavior trees for robotic assembly. In *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, pages 11511–11517.

Tulathum, P., Usawalertkamol, B., Ricardez, G.A.G., Takamatsu, J., Ogasawara, T., & Matsumoto, K. (2022). Human-robot interaction system for non-expert users in convenience stores using behavior trees. In *Proc. IEEE/SICE Int. Symp. on System Integration (SII)*, pages 1072–1077.

Vora, J. R., Helmi, A., Zhan, C., Olivares, E., Vu, T., Wilkey, M., Noregaard, S., Fitter, N. T., & Logan, S. W. (2021). Influence of a socially assistive robot on physical activity, social play behavior, and toy-use behaviors of children in a free play environment: A within-subjects study. *Frontiers in Robotics and A, I*, 8.

Wathieu, A., Groechel, T.R., Lee, H.J., Kuo, C., & Matarić, M.J. (2022). Re: Bt-espresso: Improving interpretability and expressivity of behavior trees learned from robot demonstrations. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 11518–11524. IEEE.

Wu, R., Kortik, S., & Santos, C.H. (2021). Automated behavior tree error recovery framework for robotic systems. In *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, pages 6898–6904.

Younes, H.L.S., & Littman, M.L. (2004). PPDDL 1.0: An extension to PDDL for expressing planning domains with probabilistic effects. *Techn. Rep. CMU-CS-04-162*, 2:99.

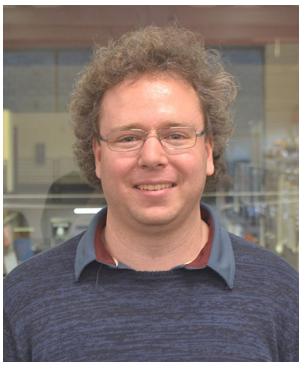
Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Emily Scheide is a Ph.D. candidate at the Collaborative Robotics and Intelligent Systems (CoRIS) Institute at Oregon State University. She received her M.S. degree in Robotics at Oregon State University and B.S. degree in Physics at the University of California, Davis. Emily's primary research focus is online and offline behavior tree synthesis and adaption with the goal of more accessibly providing robust autonomy capabilities for the robotic exploration of unstructured environments.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.



Graeme Best is a Lecturer in the Robotics Institute at the University of Technology Sydney (UTS). His research focuses on developing autonomy for multi-robot systems in challenging outdoor environments, with contributions ranging from new fundamental algorithms to full-scale system demonstrations. He completed postdoctoral studies at Oregon State University, where he participated in the DARPA Subterranean Challenge with Carnegie Mellon University. He earned his Ph.D. (2019) from The University of Sydney and his B.Eng. and B.Sc. (2014) from Monash University. Dr. Best is a recipient of the 2023 Google Research Scholar Program Award and multiple Best Paper awards and nominations at international robotics conferences, including RSS, ICRA, and IROS.



Geoffrey A. Hollinger is a Professor in the Collaborative Robotics and Intelligent Systems (CoRIS) Institute and the School of Mechanical, Industrial & Manufacturing Engineering at Oregon State University. His research focuses on decision-making algorithms for robotic systems operating in unstructured environments (e.g., underwater, aerial, and subter ranean). He has previously held research positions at the University of Southern California, Intel Research, the University of Pennsylvania's GRASP Laboratory, and NASA's Marshall Space Flight Center. He received his Ph.D. (2010) and M.S. (2007) in Robotics from Carnegie Mellon University and his B.S. in General Engineering along with his B.A. in Philosophy from Swarthmore College (2005). He is a recipient of the ONR YIP award (2017) and the NSF CAREER award (2019), and he has served as associate editor for the Autonomous Robots and IEEE Robotics and Automation Letters journals.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com